RACKET

EVALUARE

1) aplicativă

(f (+ 1 2) 3)

3

(1 / 0)

or, and

(or #f (< (1 / 0) 2))

(let ... (par 3)

2) leneșă

f (1 + 2) 3

3

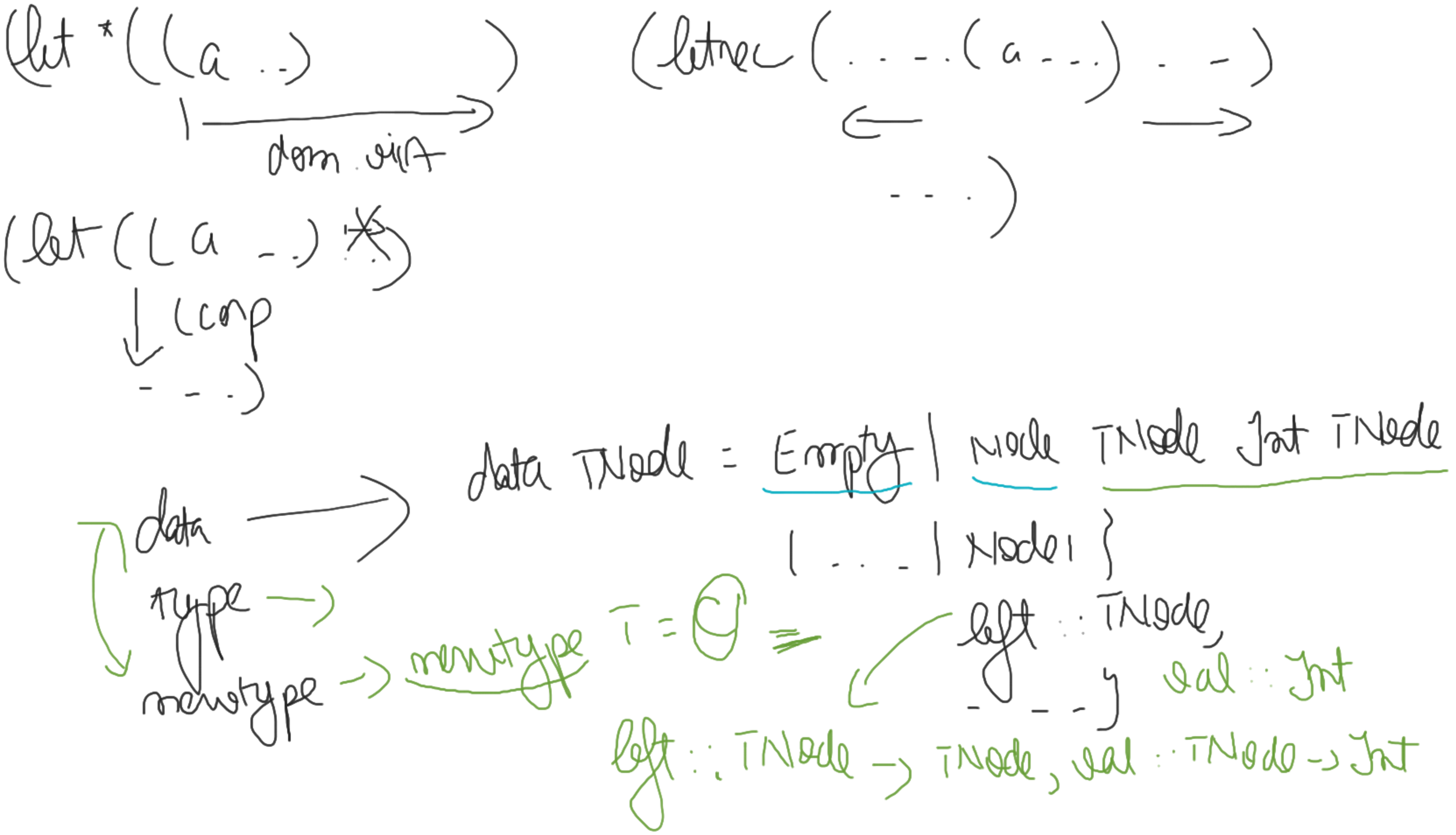
f x y : x

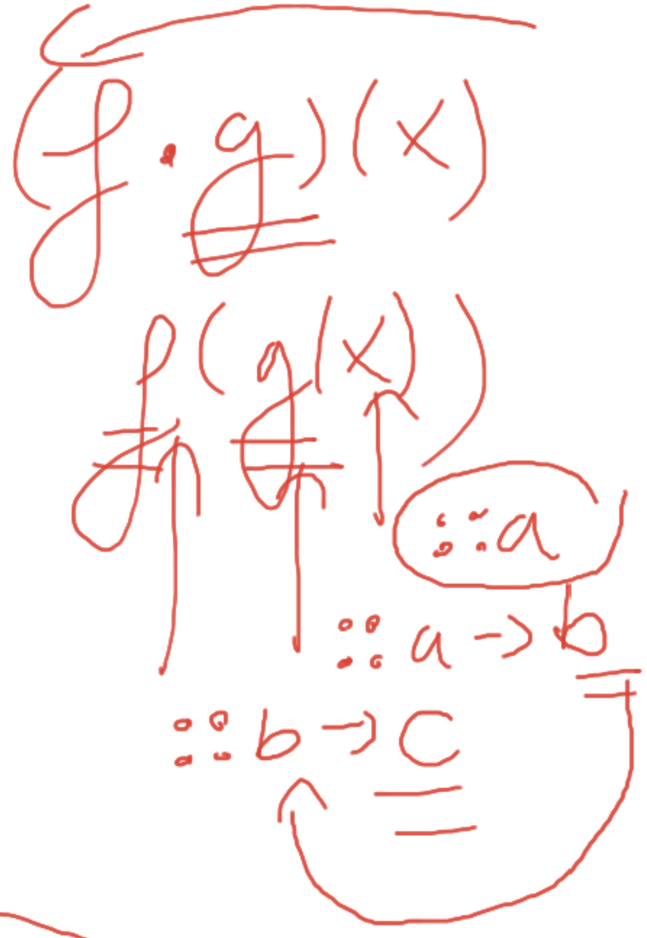
let x = 1 + 2

y = 3

z = 1 / 0

in x + y


let *((a ..)
dom. diff
(letrec (. . . . (a . . .) . .)
. . .)
(let ((a _ .) ...)
(comp
. . .)
data
type ->
newtype ->
data TNode = Empty | Node TNode Int TNode
| . . . | Node1 }
newtype T = C {
left :: TNode,
_ _ _ } val :: Int
left :: TNode -> TNode, val :: TNode -> Int



$(f . g) :: a \to c$

$(.) :: (b \to c) \to (a \to b) \to a \to c$

$(\$) :: (a \to b) \to a \to b$

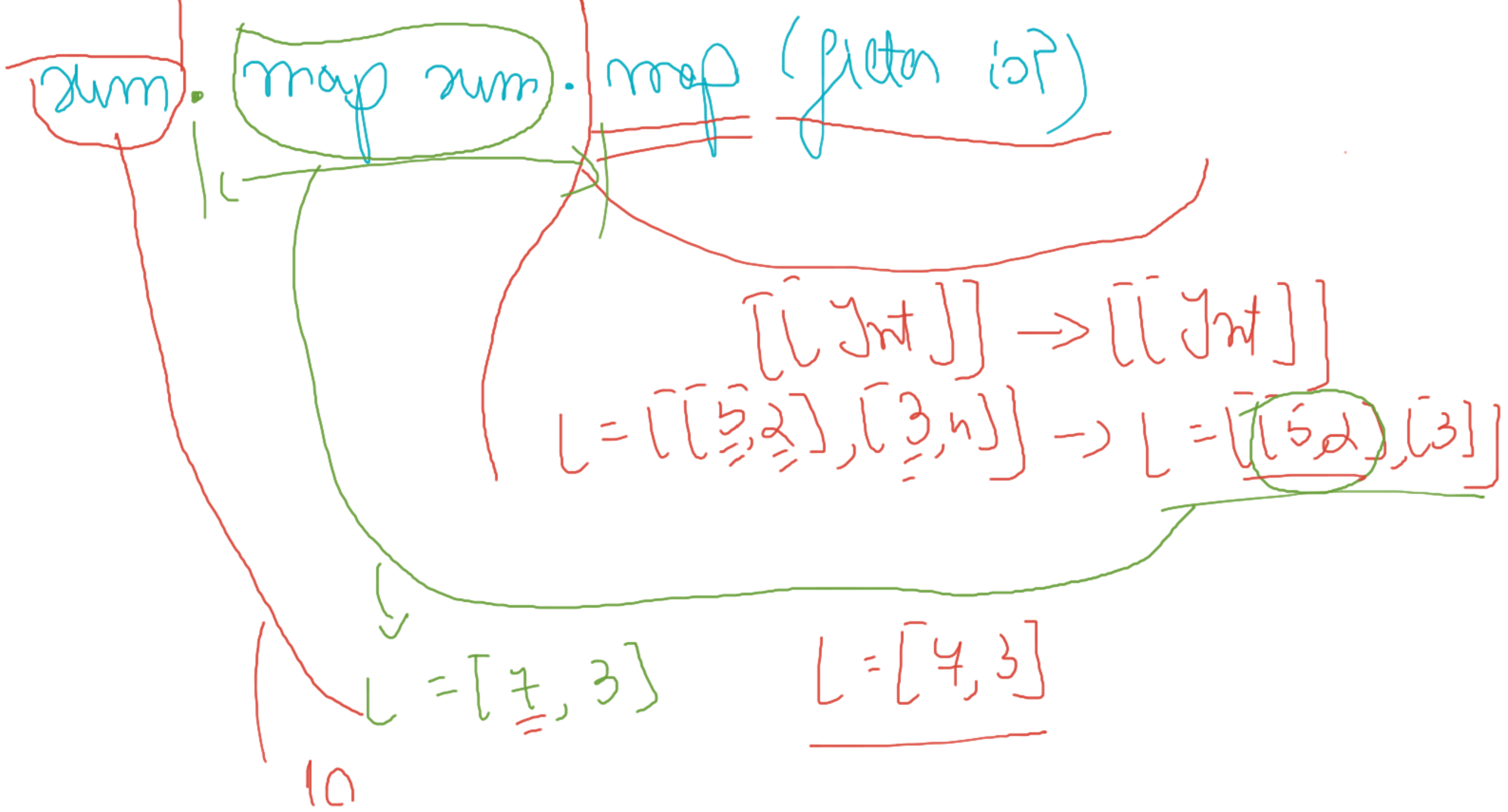
length $ map f L

filter :: (a -> Bool) -> [a] -> [a]

f

L

res


sum . map sum . map (filter isP)
[[Int]] -> [[Int]]
L = [[5,2],[3,4]] -> L = [[5,2],[3]]
L = [7,3]
L = [4,3]
10



($) :: (a -> b) -> a -> b

map x . y :: a -> b

z :: a

(.) :: (b0 -> c0) -> (a0 -> b0) -> a0 -> c0

map x :: b0 -> c0 = b0 -> b

y :: a0 -> b0 = a -> b0

map x . y = a0 -> c0 = a -> b

=> a0 = a

c0 = b

$map :: (a_1 \to b_1) \to [a_1] \to [b_1]$

$x :: a_1 \to b_1$

$map\ x :: [a_1] \to [b_1] = b_0 \to b \Rightarrow b_0 = [a_1]$, $b = [b_1]$

$x :: a_1 \to b_1$

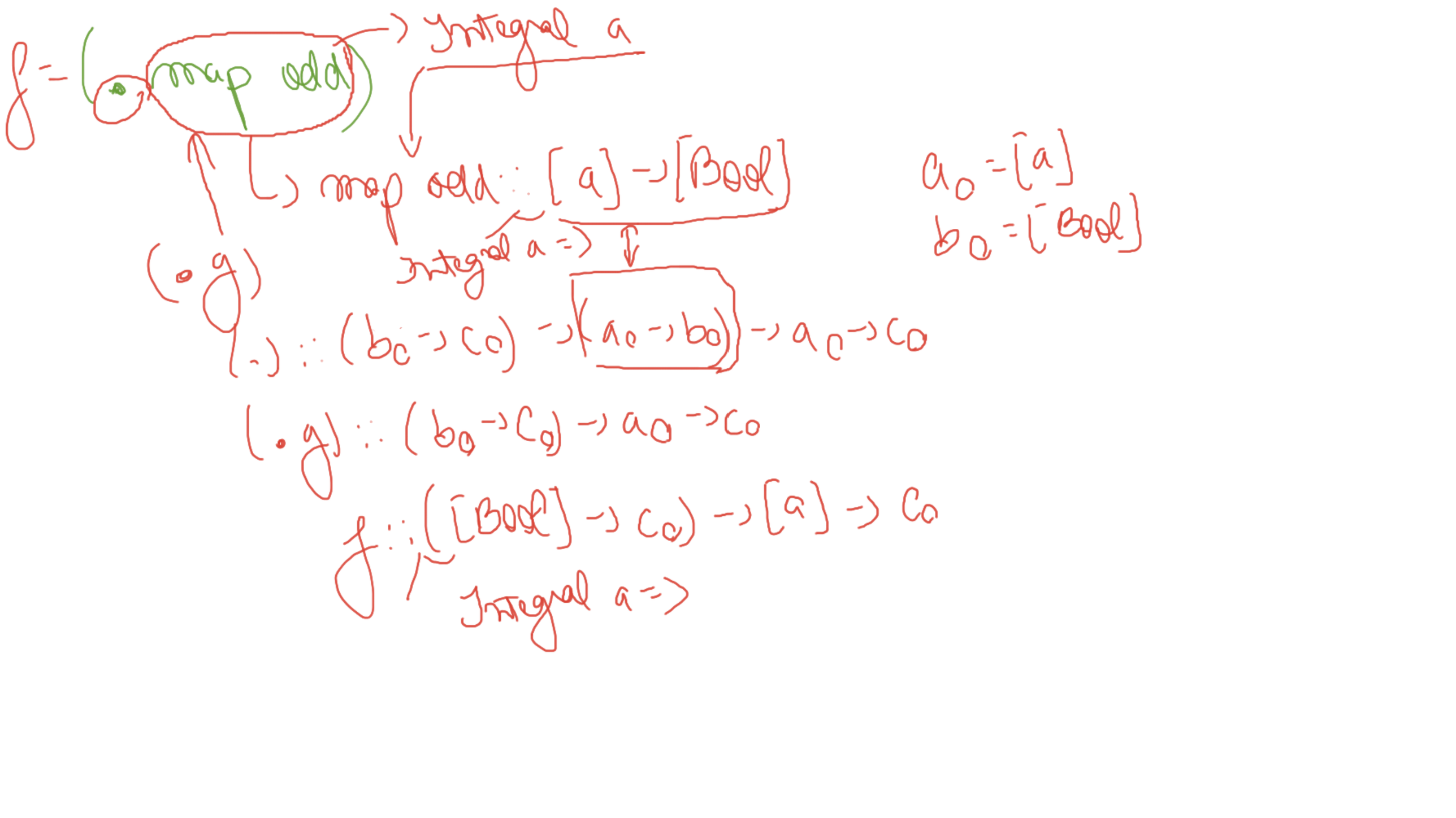
$y :: a \to b_0 = a \to [a_1]$

$z :: a$

$f\ x\ y\ z :: b = [b_1]$

$f :: (a_1 \to b_1) \to (a \to [a_1]) \to a \to [b_1]$

$(a \to b) \to (c \to [a]) \to c \to [b]$


f = (map odd)
Integral a
map odd :: [a] -> [Bool]
a0 = [a]
b0 = [Bool]
(.g)
Integral a =>
(.) :: (b0 -> c0) -> (a0 -> b0) -> a0 -> c0
(.g) :: (b0 -> c0) -> a0 -> c0
f :: ([Bool] -> c0) -> [a] -> c0
Integral a =>

a -> a -> a

1) a -> a -> a

2) (a -> a) -> a

3) a -> (a -> a)



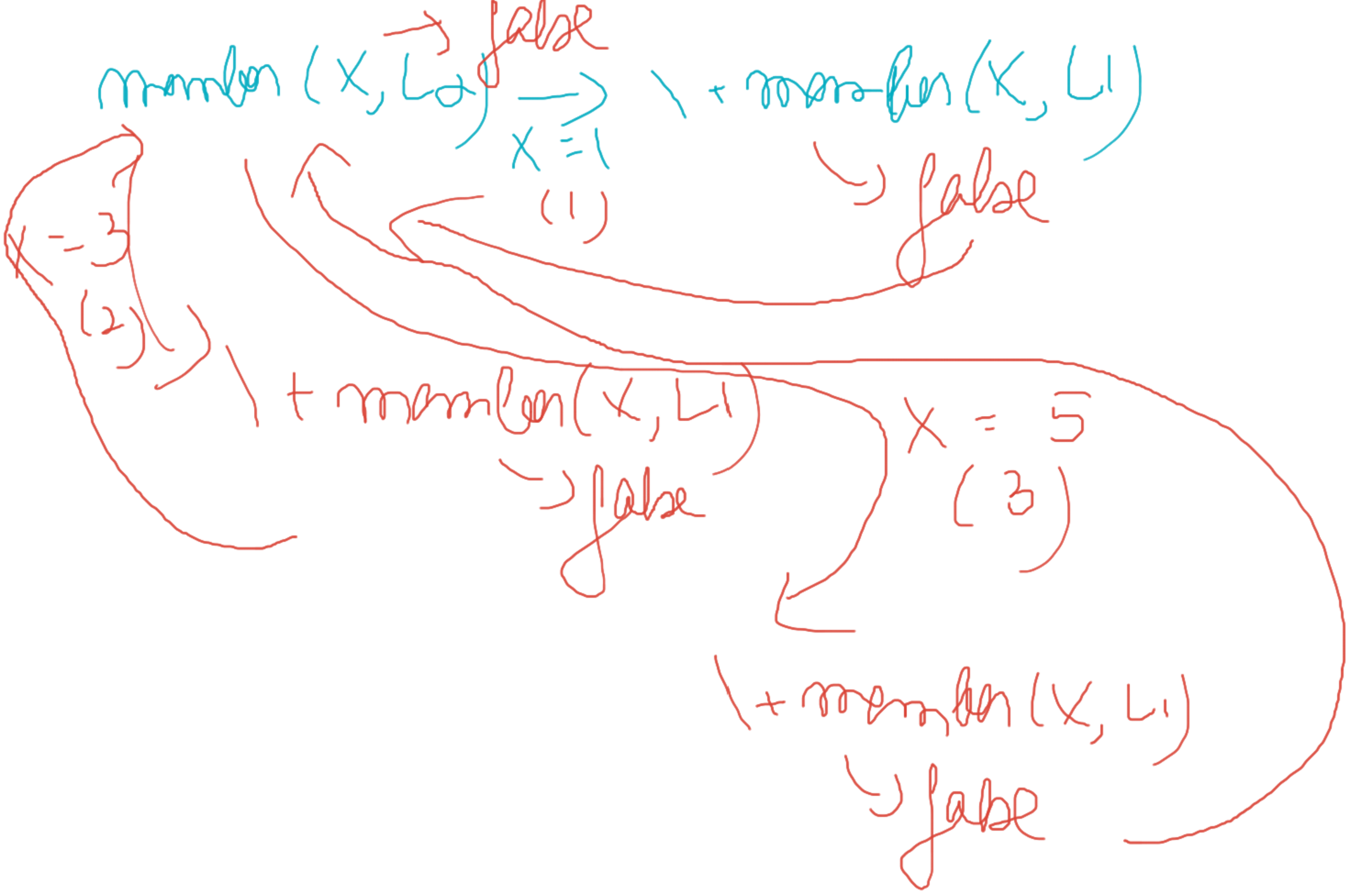
(1)

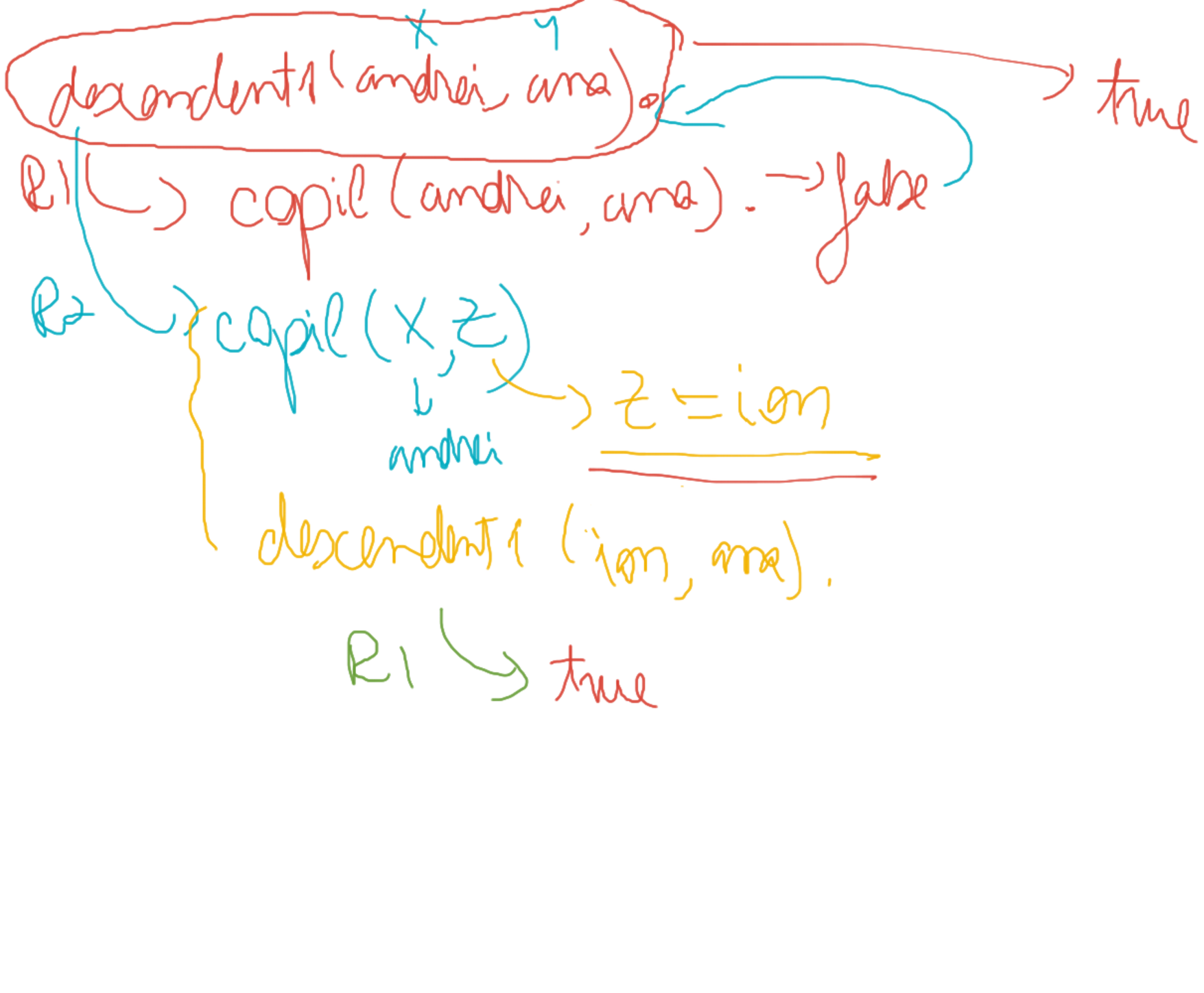
class (Num a) => Fractional a where

(2)

(/) ...

(c) ...


false
X = 1
(1)
false
-3
(2)
X = 5
(3)
false
false


descendent1(andrei, ana)
X
Y
true
R1
copil(andrei, ana). → false
R2
copil(X, Z)
X
andrei
Z = ion
descendent1(ion, ana).
R1
true

descendente2 (andrei, ana)

R1 → descendente2 (z, ana)

(R1) → descendente2 (z', ana), → ...

(R1) → ...